

Intermediate GitHub & DVC Project Tasks

Problem Statement 1 – Collaborative Flask Development with Git and GitHub

You are part of a software development team working on a shared Flask web application hosted on GitHub. The goal of this task is to help students understand how to work collaboratively using Git, GitHub, and multiple branches — and how to identify and resolve merge conflicts when multiple team members edit the same file.

- **Clone the Remote Repository:** Each student must clone the shared GitHub repository to their local system to begin working on the project. This establishes a local copy connected to the remote main branch.
- **Create a New Branch for Each Developer:** Each student creates and switches to a new feature branch (e.g., feature-login, feature-dashboard, feature-api) to isolate their changes and follow the Git branching workflow.
- **Modify the Same File (housepk_app.py):** Every student edits the same Flask application file and adds new routes, functions, or print statements that intentionally overlap or modify the same lines of code to simulate real-world conflicts.
- **Stage, Commit, and Push Changes:** After making changes, each student commits their local modifications and pushes their feature branch to the remote repository on GitHub.
- **Merge Branches into Main:** One student (acting as the team lead) will merge each feature branch sequentially into the main branch. Conflicts will occur when Git detects that multiple branches modified the same section of the file differently.
- **Identify and Resolve Merge Conflicts:** Students will open the conflicted file, review Git's conflict markers, and manually edit the code to create a clean, functional version that combines contributions from different team members.
- **Stage, Commit, and Push Resolved Code:** After resolving conflicts, the team lead stages and commits the resolved files, then pushes the updated main branch to the remote repository.
- **Pull the Latest Code:** All students pull the latest version of the main branch to synchronize their local copies with the resolved, updated codebase.

Problem Statement 2 – DVC-Based ML Pipeline and Flask Deployment

You are a data scientist working on a house price prediction system using the Pakistan House Price Dataset from Kaggle. The objective of this task is to implement an end-to-end reproducible machine learning workflow using DVC (Data Version Control) integrated with Git and GitHub, followed by deployment of the trained model using Flask.

- **Set Up Project Repository:** Create a new project folder and initialize it as a Git repository. Push the initial setup to a GitHub repository that will serve as the central project workspace.
- **Integrate DVC for Data Versioning:** Initialize DVC in the project and configure a storage remote (local, Google Drive, or S3). Download the Pakistan House Price Dataset from Kaggle, store it in a data/ directory, and track it using DVC to enable version control for large files.
- **Create Configuration Files (params.yaml):** Define key parameters such as model type, random seed, test size, and hyperparameters inside params.yaml. This will make the training process

reproducible and parameter-driven.

- **Develop Training Pipeline:** Implement a DVC pipeline by defining a `dvc.yaml` file that specifies dependencies (data, code, parameters) and outputs (trained model). The pipeline should automate model training, saving artifacts like the model file and metrics.
- **Version Control Model and Outputs:** After running the pipeline, commit changes to Git and push both Git and DVC tracked artifacts (data and models) to the respective remote storages. This ensures every experiment version is reproducible from Git and DVC history.
- **Deploy Model with Flask:** Create a Flask web application that loads the trained model, accepts user input (house features), and returns predicted prices. Ensure the app can fetch the latest model version by syncing files from GitHub or DVC storage.
- **Test and Validate Deployment:** Run the Flask server locally, test it using sample input data, and confirm that predictions are generated correctly using the latest trained model.
- **Push Final Code and Artifacts to GitHub:** Commit and push all final code, configuration, and pipeline files (`params.yaml`, `dvc.yaml`, and `dvc.lock`) to the GitHub repository to make the entire workflow reproducible by others.